

제3장 : 무작위성과 확률

이 장에서는 확률과 무작위성 개념을 게임 AI에 적용하는 방법을 살펴보겠습니다. 이 장에서는 Unity3D 자체보다는 게임 AI에서의 무작위성 활용을 더 중점적으로 다루므로, 여기에서 소개하는 개념들은 어떤 게임 개발 미들웨어나 기술 프레임워크에도 적용할 수 있습니다. 데모는 Unity3D에서 Mono C#을 사용하여 진행하겠지만, Unity3D 엔진과 에디터 자체의 구체적인 기능에 대해서는 많이 다루지 않겠습니다.

게임 개발자는 AI 캐릭터와 더 넓은 게임 세계의 행동에 약간의 불확실성을 부여하기 위해 확률을 사용합니다. 무작위성은 인공지능을 더 현실적이고 자연스럽게 보이게 하며, 의도적인 예측 가능성이 필요하지 않은 모든 경우에 완벽한 “양념” 역할을 합니다.

이 장에서는 다음 주제들을 다룹니다:

- 유니티에서의 무작위성 소개
- 확률의 기초 학습하기
- 게임 내 확률의 추가 예제 탐구하기
- 슬롯머신 만들기

기술 요구사항

이 장을 위해서는 Unity3D 2022만 있으면 됩니다. 이 장에서 설명하는 예제 프로젝트는 책 저장소의 **Chapter 3** 폴더에서 찾을 수 있습니다. <https://github.com/PacktPublishing/Unity-Artificial-Intelligence-Programming-Fifth-Edition/tree/main/Chapter03> .

Unity에 무작위성 도입하기

게임 디자이너와 개발자는 캐릭터의 의사 결정 결과를 변경하여 게임과 캐릭터를 더 사실적으로 만들기 위해 게임 AI에 무작위성을 사용합니다.

전형적인 축구 게임을 예로 들어 보겠습니다. 축구 게임의 규칙 중 하나는 상대 팀 선수가 공을 빼앗으려다 파울을 범하면 해당 팀에게 직접 프리킥(direct free kick)을 부여하는 것입니다. 그러나 파울이 발생할 때마다 항상 프리킥을 주는 대신, 게임 개발자는 확률을 적용하여 모든 파울의 98%에 대해서만 직접 프리킥을 보상으로 주도록 할 수 있습니다.

결국 현실에서도 심판은 가끔 실수를 하기 마련입니다. 이러한 간단한 변경의 결과로 플레이어는 보통 예상대로 직접 프리킥을 얻게 됩니다. 하지만 나머지 2%의 상황이 발생했을 때, 게임은 양 팀 모두에게 더 큰 감정적 피드백을 제공합니다(다른 사람과 플레이한다고 가정할 때, 한 플레이어는 기뻐할 것이고 다른 플레이어는 가상의 심판에게 항의할 것입니다).

물론 무작위성이 항상 AI의 바람직한 특전인 것은 아닙니다. 도입부에서 예상했듯이, 어느 정도의 예측 가능성은 플레이어가 AI 패턴을 학습할 수 있게 해 주며, AI 패턴을 이해하는 것은 종종 게임플레이의 주

요 요소가 됩니다. 예를 들어 스텔스 게임(stealth game)에서는 플레이어가 몰래 잠입할 경로를 찾기 위해 적 경비병의 이동 경로를 파악하는 것이 필수적입니다. 혹은 다크 소울(Dark Souls) 같은 게임의 보스를 디자인해야 한다고 상상해 보십시오.

거대 보스의 공격 패턴을 학습하는 것은 플레이어의 주된 무기이자 보스전을 마스터하기 위한 유일하고 올바른 방법입니다. 언제나 그렇듯, 여러분은 게임 디자인의 북극성(지향점)을 따라야 합니다. 즉, 플레이어에게 재미있는 것만 하십시오. 무작위성을 추가하는 것이 플레이어에게 좌절감만 안겨준다면, 예외 없이 이를 제거해야 합니다.

하지만 어떤 경우에는 약간의 무작위성이 유용하며, 도박형 미니게임과 같은 일부 게임에서는 필수 전제 조건입니다. 이러한 경우, 컴퓨터는 어떻게 무작위 값(random values)을 생성할 수 있을까요? 그리고 더 중요한 것은, 유니티에서 난수(random numbers)를 어떻게 사용할 수 있을까요?

컴퓨터 과학에서의 무작위성

컴퓨터는 결정론적 기계(deterministic machines)입니다. 설계상 컴퓨터에 프로그램 코드와 데이터 형태의 동일한 입력을 여러 번 주면 항상 동일한 출력을 반환합니다. 그렇다면 어떻게 프로그램이 예측 불가능하고 무작위적인 출력을 반환하게 할 수 있을까요?

진정한 난수(genuinely random numbers)가 필요하다면 다른 곳에서 이 무작위성을 가져와야 합니다. 그렇기 때문에 많은 고급 애플리케이션에서는 무작위성의 다양한 외부 소스를 결합하여 하나의 무작위 값으로 만들려고 시도합니다. 예를 들어 특정 간격 동안의 마우스 움직임, 인터넷 연결의 노이즈를 확인하거나, 심지어 사용자에게 키보드를 무작위로 두드리라고 요청하는 등의 방법을 사용합니다. 심지어 난수 생성을 위한 전용 하드웨어도 있습니다!

다행히 게임에서는 이렇게 완벽하게 순수한 난수가 필요하지 않으며, 난수 시퀀스처럼 보이는 시퀀스를 생성할 수 있는 더 간단한 알고리즘을 사용할 수 있습니다. 이러한 알고리즘을 의사 난수 생성기(PRNG, Pseudorandom Number Generators)라고 부릅니다. 초기 시드(seed, 씨앗값)를 사용하여 이들은 결정론적인 방식으로 진정한 난수 시퀀스의 특성에 통계적으로 근사하는 숫자 시퀀스를 생성할 수 있습니다. 단점은 동일한 시드에서 시작하면 항상 동일한 숫자 시퀀스를 얻게 된다는 것입니다.

이러한 이유로 우리는 보통 사용자가 애플리케이션을 열 때마다 항상 다르다고 생각되는 값, 예를 들어 컴퓨터가 실행된 후 경과된 시간(밀리초)이나 1970년 이후의 밀리초 단위 시간(유닉스 타임스탬프) 등으로 시드 값을 초기화합니다. 하지만 디버깅을 할 때는 매번 동일한 무작위 시퀀스를 얻을 수 있다는 점이 사실 매우 유용하다는 것을 기억하세요!

마지막으로, 일부 PRNG는 다른 것들보다 더 무작위적입니다. 암호화 프로그램을 만든다면 암호학적으로 안전한 의사 난수 생성기(CSPRNG, Cryptographically Secure Pseudorandom Number Generators)라는 덜 예측 가능한 PRNG를 찾아봐야 할 것입니다. 다행히도 게임의 경우 유니티에서 제공하는 간단한 난수 생성(RNG)만으로도 충분합니다.

Unity Random 클래스

유니티 3D 스크립트에는 무작위 데이터를 생성하는 Random 클래스가 있습니다. InitState(int seed) 함수를 사용하여 생성기 시드를 설정할 수 있습니다. 일반적으로는 동일한 값을 반복해서 시드로 설정하지 않습니다. 그렇게 하면 실행할 때마다 예측 가능한 동일한 난수 시퀀스가 생성되기 때문입니다.

하지만 사용자가 시드를 제어할 수 있게 하고 싶은 경우도 있습니다. 예를 들어, 게임을 테스트할 때나 플레이어가 특정 시드로 절차적(procedural) 맵/레벨을 생성하도록 하고 싶을 때입니다. 그런 다음 Random.value 속성을 읽어 0.0에서 1.0 사이의 난수를 얻을 수 있습니다. 이 생성기는 양끝 값을 포함(inclusive)하므로 이 속성은 0.0과 1.0을 모두 반환할 수 있습니다.

예를 들어 다음 스니펫(snippet)에서는 빨강, 초록, 파랑(RGB) 컴포넌트에 대해 0과 1 사이의 무작위 값을 선택하여 무작위 색상을 생성합니다:

```
Color randomColor = new Color(Random.value, Random.value, Random.value);
```

아주 유용하게 쓰일 수 있는 또 다른 클래스 메서드는 Range 메서드입니다:

```
static function Range (min : float, max : float) : float
```

Range 메서드를 사용하면 특정 범위에서 난수를 생성할 수 있습니다. 정수(integer) 값이 주어지면 min(포함, inclusive)과 max(제외, exclusive) 사이의 무작위 정수를 반환합니다. 따라서 min을 1로, max를 4로 설정하면 1, 2 또는 3을 얻을 수 있지만 4는 절대 나오지 않습니다. 반면에 float 값에 Range 함수를 사용하면 min과 max가 모두 포함되므로 1.0이나 4.0, 또는 그 사이의 모든 실수(float)를 얻을 수 있습니다. 유니티의 Random 클래스를 사용할 때 매개변수(parameter)가 포함인지 제외인지에 유의하십시오. 이는 버그(및 혼동)의 흔한 원인이 됩니다.

간단한 무작위 주사위 게임 (A simple random dice game)

6면체 주사위의 결과(1에서 6 사이의 무작위 정수를 생성하여 시뮬레이션함)를 맞혀야 하는 직관적인 주사위 게임을 새로운 씬(scene)에 설정해 보겠습니다. 무작위로 생성된 주사위 결과와 입력값이 일치하면 플레이어가 승리합니다. 코드는 다음 DiceGame.cs 파일과 같습니다:

```
using UnityEngine;
using TMPro;
using UnityEngine.UI;

public class DiceGame : MonoBehaviour {
    public string inputValue = "1";
    public TMP_Text outputText;
    public TMP_InputField inputField;
    public Button button;
```

```

int throwDice() {
    Debug.Log("주사위 던지는 중...");
    Debug.Log("1에서 6 사이의 난수 찾는 중...");
    int diceResult = Random.Range(1,7);
    Debug.Log($"결과: {diceResult}");
    return diceResult;
}

public void processGame() {
    inputValue = inputField.text;
    try {
        int inputInteger = int.Parse(inputValue);
        int totalSix = 0;
        for (var i = 0; i < 10; i++) {
            var diceResult = throwDice();
            if (diceResult == 6) { totalSix++; }

            if (diceResult == inputInteger) {
                outputText.text = $"DICE RESULT:
                {diceResult} \r\nYOU WIN!";
            } else {
                outputText.text = $"DICE RESULT:
                {diceResult} \r\nYOU LOSE!";
            }
        }
        Debug.Log($"6이 나온 총 횟수: {totalSix}");
    } catch {
        outputText.text = "Input is not a number!";
        Debug.LogError("입력값이 숫자가 아닙니다!");
    }
}
}

```

이전 코드에서 전체 게임을 구현하는 DiceGame 클래스를 보았습니다. 하지만 플레이어의 입력을 받고 결과를 표시할 적절한 UI 오브젝트로 씬을 설정해야 합니다:

1. 먼저 결과를 표시할 guiText를 생성해야 합니다. **Game Object | UI | Text - TextMeshPro**를 클릭합니다. 그러면 게임 씬에 New Text 텍스트가 추가됩니다.
2. 캔버스(canvas) 상단 중앙에 배치합니다.
3. 동일한 방식으로 **Game Object | UI | Button - TextMeshPro**를 선택하여 버튼을 생성하고, **Game Object | UI | Input Field - TextMeshPro**를 선택하여 입력 필드(input field)를 생성합니다.
4. 화면에 수직으로 정렬합니다.

5. 빈 게임 오브젝트를 생성하고 이름을 DiceGame으로 지정합니다. 이 시점에서 그림 3.1과 비슷한 화면이 되어야 합니다.

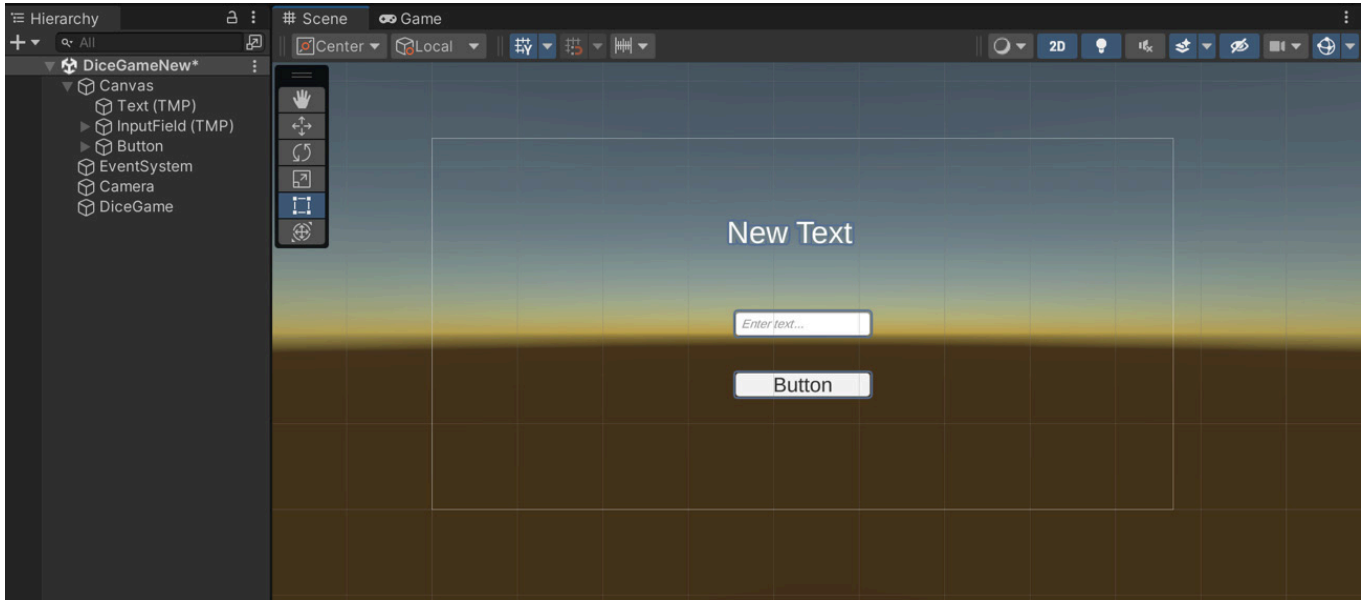


그림 3.1 – 간단한 Unity 인터페이스

6. 버튼 안의 텍스트를 선택하고 TextMeshPro 컴포넌트에서 Button을 Play!로 변경합니다.
7. New Text 텍스트를 선택하고 TextMeshPro 컴포넌트에서 Result:로 변경합니다.

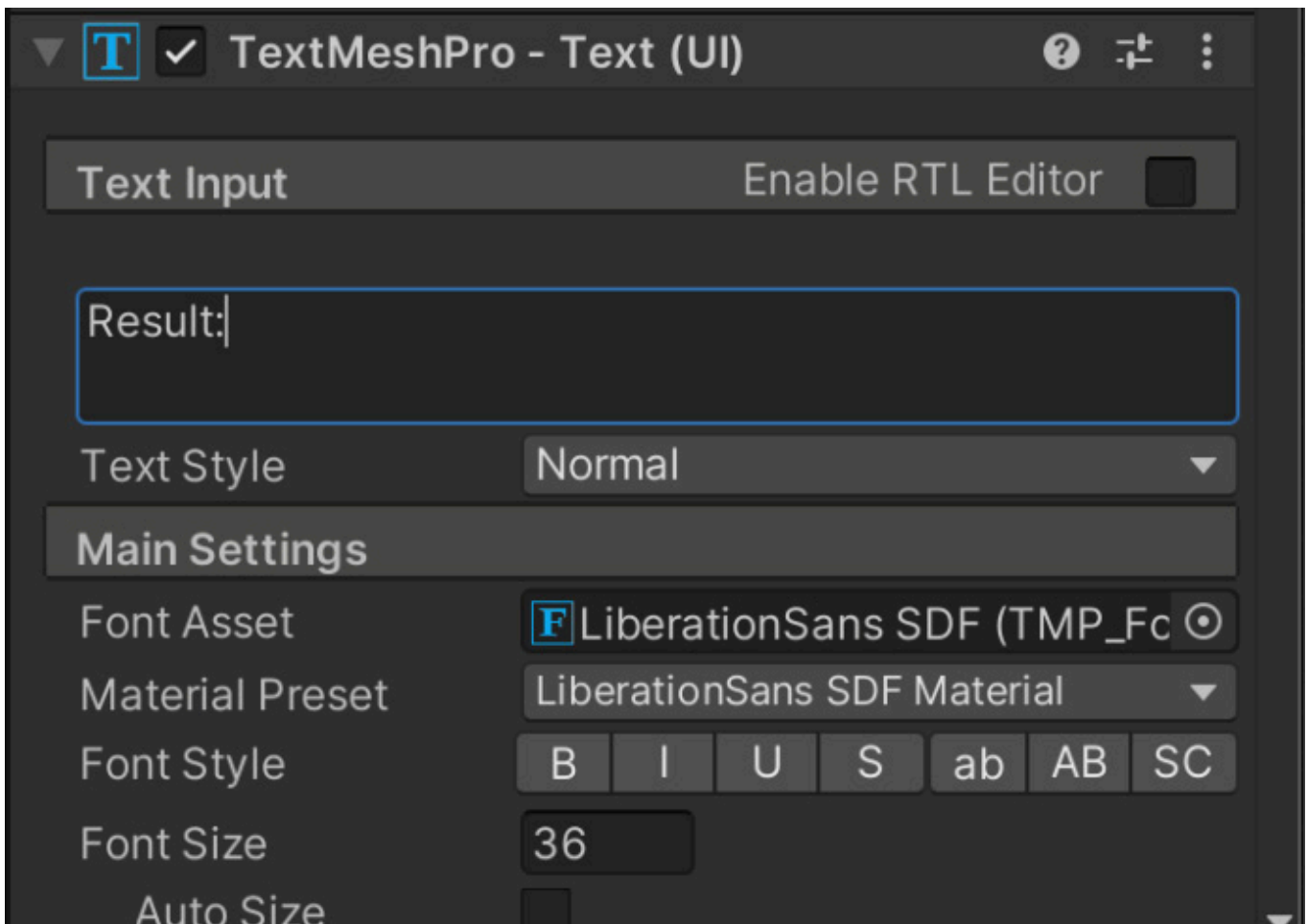


그림 3.2 – TextMeshPro 컴포넌트

8. 이제 DiceGame 컴포넌트를 DiceGame 오브젝트에 연결(attach)하고, 앞서 생성한 세 개의 UI 요소를 DiceGame 컴포넌트에 연결합니다.

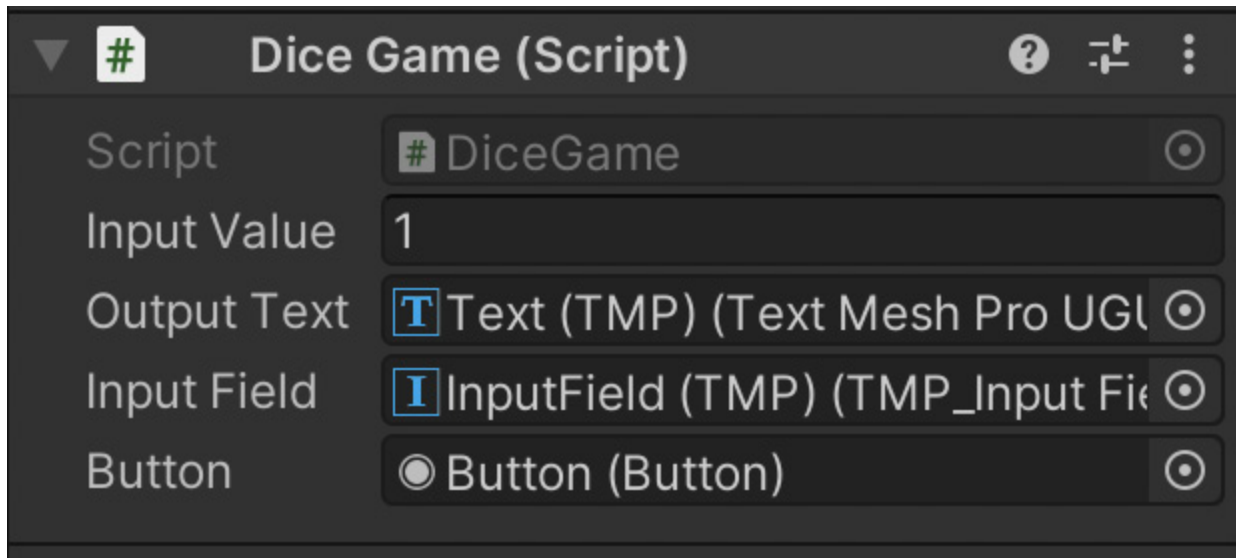


그림 3.3 – DiceGame 컴포넌트

9. 마지막으로 Button을 선택하고 Button 컴포넌트에서 onClick() 섹션을 찾습니다. None (GameObject) 필드에 DiceGame 오브젝트를 드래그 앤 드롭하고, 드롭다운 메뉴에서 DiceGame | processGame()을 선택합니다. 이렇게 하면 버튼의 클릭 이벤트에 processGame 함수가 연결됩니다.

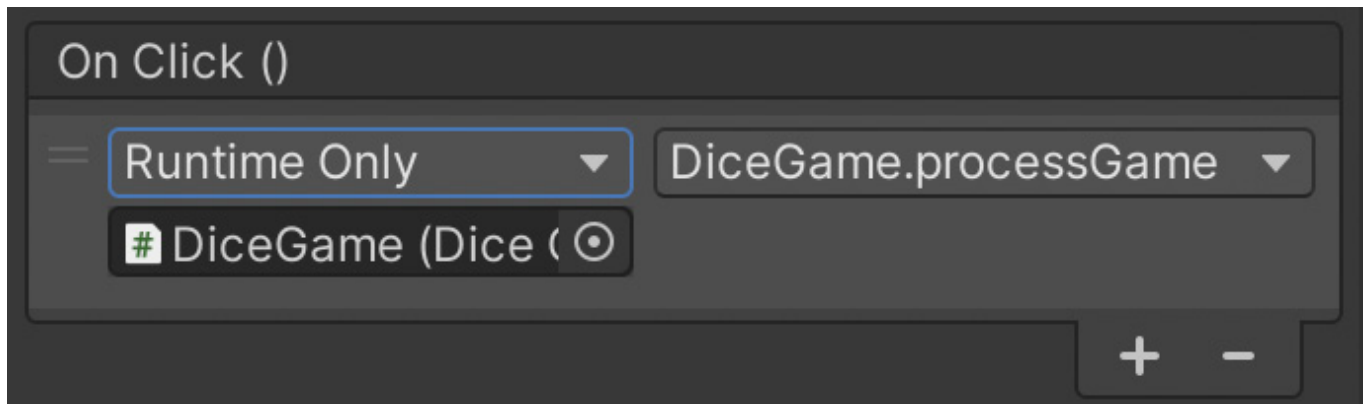


그림 3.4 – On Click 이벤트 구성

이 시점에서 게임이 준비되었을 것입니다. 유니티의 재생(Play) 버튼을 클릭하고 한번 실행해 보세요.

난수를 성공적으로 관리하려면 확률의 법칙(laws of probability)에 대한 기본적인 이해가 필요합니다. 이것이 우리가 다음 섹션에서 배울 내용입니다.

확률의 기초 학습하기 (Learning the basics of probability)

확률을 정의하는 방법에는 여러 가지가 있습니다. 확률에 대한 가장 직관적인 정의를 빈도주의(frequentism)라고 합니다. 빈도주의에 따르면 사건(event)의 확률은 무한한 횟수로 관찰을 반복했을 때 해당 사건이 발생하는 빈도(frequency)입니다. 다시 말해 주사위를 100번 던지면 평균적으로 1/6의 확률로 6이 나올 것으로 예상하며, 1,000번, 10,000번, 100만 번 던지면 1/6에 점점 더 가까워질 것입니다.

사건 A가 발생할 확률을 $P(A)$ 로 쓸 수 있습니다. $P(A)$ 를 계산하려면 관찰 가능한 모든 결과(N)와 원하는 사건이 발생하는 총 횟수(n)를 알아야 합니다.

사건 A의 확률은 다음과 같이 계산할 수 있습니다:

$$P(A) = \frac{n}{N}$$

$P(A)$ 가 사건 A가 일어날 확률이라면, 사건 A가 일어나지 않을 확률은 다음과 같습니다:

$$\bar{P} = 1 - P(A)$$

확률은 0과 1 사이의 실수(real number)여야 합니다. 확률이 0이라는 것은 원하는 사건이 일어날 가능성이 전혀 없다는 뜻이며, 반대로 확률이 1이라는 것은 사건이 확실히 발생한다는 것을 의미합니다. 결과적으로 다음의 합은 1과 같아야 합니다:

$$P(A) + \bar{P}(A) = 1$$

그러나 모든 사건이 똑같은 것은 아닙니다. 확률 계산에서 가장 중요한 개념 중 하나는 독립 사건(independent events)과 상관 사건(non-independent events)의 개념입니다. 이것이 다음 섹션의 주제입니다.

독립적 사건과 상관관계가 있는 사건(Independent and correlated events)

확률에서 또 다른 중요한 개념은 특정 사건이 발생할 확률이 어떤 식으로든 다른 사건에 의존하는지 여부입니다. 예를 들어, 6면체 주사위를 두 번 던져 더블 식스(6이 두 번 나옴)가 나오는 경우를 생각해 보세요. 각각의 주사위 던지기는 독립 사건으로 볼 수 있습니다. 주사위를 던질 때마다 각 면이 나올 확률은 1/6이며, 두 번째 주사위를 굴린 결과는 첫 번째 굴린 결과에 따라 달라지지 않습니다. 반면에 동일한 카드 덱(deck)에서 두 장의 에이스(ace)를 뽑는 경우, 각 뽑기는 다른 뽑기와 독립적이지 않습니다. 첫 번째 사건에서 에이스를 뽑았다면, 덱에 에이스가 한 장 적어지고(전체 카드 수도 한 장 적어지므로) 두 번째로 에이스를 얻을 확률이 달라집니다.

사건의 독립성은 일부 계산을 크게 단순화하기 때문에 매우 중요합니다. 예를 들어, 사건 A나 사건 B 둘 중 하나가 일어날 확률을 알고 싶다고 가정해 봅시다. A와 B가 두 개의 독립 사건이라면 A와 B의 확률을 더할 수 있습니다:

$$P(A \text{ or } B) = P(A) + P(B)$$

같은 방식으로 A와 B가 모두 발생할 확률을 알고 싶다면 각각의 확률을 함께 곱할 수 있습니다:

$$P(A \text{ and } B) = P(A) \cdot P(B)$$

예를 들어 두 개의 주사위를 던져 두 개의 6이 나올 확률을 알고 싶다면 1/6에 1/6을 곱하여 정확한 확률인 1/36을 얻을 수 있습니다.

조건부 확률 (Conditional probability)

이제 다른 예를 생각해 보겠습니다. 여전히 두 개의 주사위를 던지지만, 이번에는 두 주사위에 나온 숫자의 합이 2와 같을 확률에 관심이 있습니다. 이 합계를 얻는 방법은 1 더하기 1 한 가지뿐이므로, 이 확률은 두 주사위에서 같은 숫자가 나올 확률과 같습니다. 이 경우에도 여전히 1/36이 될 것입니다.

하지만 두 주사위에 나온 숫자의 합이 7이 되는 경우는 어떨까요? 다음 표에서 볼 수 있듯이 총 7을 얻을 수 있는 방법은 총 6가지가 있습니다:

Dice 1	Dice 2
1	6
2	5
3	4
4	3
5	2
6	1

그림 3.5 – 두 개의 주사위를 던졌을 때 나올 수 있는 결과

이 경우에는 일반 확률 공식을 사용해야 합니다. 앞의 표에서 총합이 7이 되는 결과가 6개 있음을 알 수 있습니다. 2개의 주사위에 대한 전체 가능한 결과가 36가지라는 것을 알고 있으므로, 최종 확률을 6/36, 단순화하여 1/6(16.7%)로 빠르게 계산할 수 있습니다.

조작된 주사위

이제 우리가 전혀 정직하지 않았다고 가정하고, 주사위가 조작되어 숫자 6이 적힌 면이 위를 향해 떨어질 확률이 두 배가 되었다고 가정해 봅시다. 6이 나올 확률을 두 배로 늘렸으므로, 대략 $1/3(0.34)$ 정도로 6이 나올 확률을 두 배로 늘려야 하며, 결과적으로 나머지 확률은 남은 다섯 면에 균등하게 분산됩니다(각각 0.132).

이 조작된 주사위 알고리즘은 다음과 같은 방법으로 구현할 수 있습니다. 먼저 1에서 100 사이의 난수를 생성합니다. 그런 다음 무작위 값이 1에서 35 사이에 있는지 확인합니다. 그렇다면 알고리즘은 6을 반환하고, 그렇지 않으면 1에서 5 사이의 무작위 주사위 값을 얻습니다(이 값들은 동일한 확률을 갖기 때문입니다).

이를 위해 DiceGameLoaded라는 새 클래스를 만듭니다. 이 게임은 DiceGame과 동일하지만 한 가지 중요한 차이점이 있습니다. throwDice 함수가 다음과 같이 변경됩니다:

```
int throwDice() {
    Debug.Log("주사위 던지는 중...");
    int randomProbability = Random.Range(0, 100);
    int diceResult = 0;

    if (randomProbability < 35) {
        diceResult = 6;
    } else {
        diceResult = Random.Range(1, 5);
    }

    Debug.Log("결과: " + diceResult);
    return diceResult;
}
```

이 새 버전의 게임을 시도해 보려면, DiceGame 오브젝트에 있는 DiceGame 컴포넌트를 이 DiceGameLoaded 컴포넌트로 교체하고, 앞서 했던 것처럼 onClick 버튼 이벤트를 다시 바인딩(rebind)하십시오. 주사위를 여러 번 던져서 새롭게 조작된 주사위 알고리즘을 테스트해 보면, 6이라는 값이 평소보다 더 많이 나오는 것을 알 수 있습니다.

보시다시피 코드는 조작되지 않은 주사위와 매우 유사합니다. 그러나 이번에는 원래보다 6이 훨씬 더 많이 나오는 불공정한 주사위를 던지고 있습니다. 먼저 0에서 100 사이의 난수를 선택하고, 숫자가 35보다 작으면 6을 반환합니다. 그렇지 않으면 1에서 5 사이의 난수를 선택합니다. 따라서 35%의 확률로 6이 나오고 나머지 75%의 확률을 5로 나눈 대략 15%의 확률로 다른 모든 숫자가 나옵니다.

게임에서 플레이어에게 더 흥미진진하고 재미있는 경험을 제공하는 것이 목표라면, 그것은 속임수(cheating)가 아니라는 점을 기억하세요!

게임 내 확률의 추가 예제 탐구하기 (Exploring more examples of probability in games)

이 섹션에서는 비디오 게임에서 확률과 무작위성이 가장 흔하게 응용되는 몇 가지 사례를 살펴보겠습니다.

캐릭터의 성격 (Character personalities)

확률과 무작위성은 단지 주사위에만 적용되는 것이 아닙니다. 게임 내 캐릭터의 특성을 명시하기 위해 확률 분포를 사용할 수도 있습니다. 예를 들어, 지역 정부를 위한 인구 관리 게임 기획안을 설계했다고 가정해봅시다. 이 게임에서는 과세와 글로벌 인재 유치, 이민과 사회적 결속 같은 문제를 다루고 시뮬레이션해야 합니다. 우리의 기획안에는 세 가지 유형의 캐릭터, 즉 노동자(worker), 과학자(scientist), 전문직(professional)이 있습니다. 이들이 각자의 특정 작업을 수행하는 효율은 다음 표와 같이 정의됩니다.

Characters	Construction	Research and Development	Corporate jobs
Worker	95	2	3
Scientist	5	85	10
Professional	10	10	80

그림 3.6 – 각 캐릭터가 각 작업을 수행하는 효율성

이 시나리오를 어떻게 구현할 수 있는지 살펴보겠습니다. 플레이어가 증가한 인구를 수용하기 위해 새 집을 지어야 한다고 해보겠습니다. 주택 건설을 완료하려면 1,000 단위의 작업량(workload)이 필요합니다. 앞서 표에 있던 값을 특정 작업에 대해 단위(unit) 유형당 초당 수행할 수 있는 작업량으로 사용합니다.

따라서 1명의 노동자로 집을 짓는다면 건설을 완료하는 데 약 10초($1000/95$)만 걸리지만, 과학자로 지으려고 한다면 3분 이상($1000/5 = 200$ 초)이 걸립니다. 연구 개발이나 기업 업무 등 다른 작업에서도 마찬가지입니다. 물론 이러한 요소들은 게임이 진행됨에 따라 나중에 조정되거나 강화될 수 있으며, 일부 초급 작업을 더 간단하게 만들고 시간이 덜 걸리게 할 수 있습니다.

그런 다음, 특정 유닛 유형이 발견할 수 있는 특수 아이템을 도입합니다. 특정 유닛이 작업을 마칠 때마다 이 아이템을 매번 나눠주고 싶지는 않습니다. 대신 서프라이즈로서 플레이어에게 보상하고 싶습니다. 그래서 다음 표에 설명된 대로 유닛 유형에 따라 이러한 아이템을 찾을 확률을 연관시킵니다:

Special items	Worker	Scientist	Professional
Raw materials	0.3	0.1	0.0
New tech	0.0	0.3	0.0
Bonus	0.1	0.2	0.4

그림 3.7 – 유닛 유형별로 특정 물건을 찾을 확률

앞의 표는 노동자가 공장이나 집을 지을 때마다 30%의 확률로 원자재를 찾고, 10%의 확률로 보너스 수입을 얻는 것을 보여줍니다. 이를 통해 플레이어는 일부 작업을 완료한 후 앞으로 나올 수 있는 보상을 기대하게 되며, 사건의 결과를 모르기 때문에 게임이 더욱 재미있어집니다.

인지된 무작위성 (Perceived randomness)

무작위성의 중요한 측면 중 하나는 인간이 진정한 무작위성을 이해하는 데 끔찍하게 서툴다는 것입니다. 오히려 우리 인간은 무작위적 결과에 대해 이야기할 때 균등하게 분포된(equally distributed) 결과를 생각합니다. 예를 들어 리그 오브 레전드(League of Legends)와 같은 대규모 멀티플레이어 온라인 배틀 아레나(MOBA) 게임을 상상해 보세요. 엄청난 데미지를 주지만 성공률이 50%에 불과한 능력을 가진 영웅이 있다고 가정해 봅시다. 플레이어가 그런 영웅으로 게임을 시작했는데, 운이 나빠서 그 능력이 5번 연속으로 빗나갔습니다. 그 플레이어의 입장이 되어 보세요. 컴퓨터가 속임수를 쓰거나 뭔가 잘못되었다고 생각하지 않겠습니까?

하지만 5번 연속 빗나갈 확률은 $1/32$ 입니다. 이는 약 3.1%로 포커의 5장 카드 딜에서 쓰리카드(트리플)가 나올 확률(약 2.1%)보다 높습니다. 희박하지만 가능한 일입니다. 게임이 완벽한 난수 생성기를 사용한다면 비교적 자주 이런 상황이 발생할 수 있습니다.

다른 방식으로 이야기해 봅시다. 빗나감(M)과 명중(H)의 시퀀스가 주어졌을 때, HHHHHMMM과 HMHMHMHM 중 어느 시퀀스가 더 무작위적이라고 생각하십니까? 장담하건대 빗나감과 명중을 교차시킨 두 번째 시퀀스일 것입니다. 자연적으로 발생할 확률이 완전히 똑같음에도 불구하고, 첫 번째 시퀀스(명중과 빗나감이 스트라이크로 예쁘게 그룹화된 곳)보다 두 번째 시퀀스가 더 무작위적인 것처럼 느껴집니다.

핵심은, 때때로 플레이어의 몰입(engagements)을 위해 게임이 진정한 무작위성보다 더 무작위처럼 느껴지는 것을 얻기 위해 무작위성을 미세하게 조정(tweak)해야 한다는 것입니다. 비디오 게임은 여러 가지 방법으로 이를 수행합니다. 가장 일반적인 방법은 무작위로 인식되어야 하는 값의 발생 횟수를 추적하는 것입니다.

따라서 예를 들어, 우리는 영웅 능력의 명중과 빗나감 횟수를 추적하고, 둘 사이의 비율이 50%라는 이론적 비율에서 너무 멀어지는 것을 보게 될 때 (예를 들어, 75%의 빗나감(또는 명중)이 발생했을 때), 강제로 명중(또는 빗나감)을 발생시켜 비율을 재조정합니다.

확률을 적용한 FSM (FSM with probability)

2장, '유한 상태 기계'에서 간단한 switch 문이나 FSM 프레임워크를 사용하여 FSM을 구현하는 방법을 보았습니다. 우리는 주어진 조건의 참 또는 거짓(true or false) 값에 전적으로 기초하여 어떤 상태를 실행할지 결정했습니다. 잠시 AI가 제어하는 탱크 엔티티의 FSM으로 돌아가 보겠습니다:

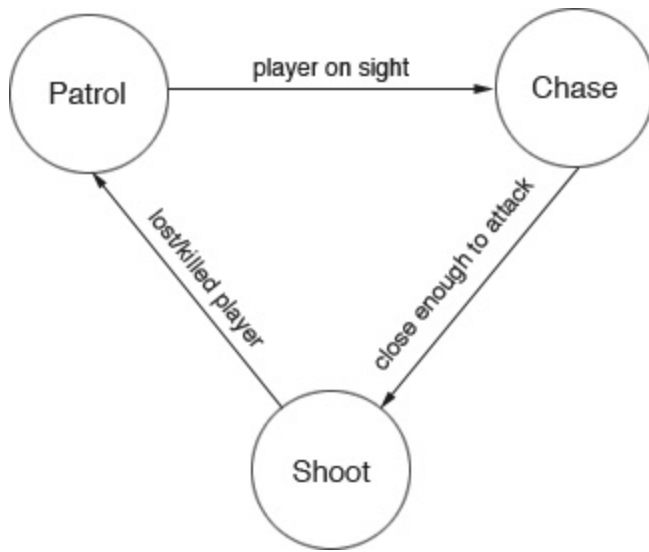


그림 3.8 – 탱크 AI FSM

예시를 위해, 탱크가 특정 조건을 충족할 때마다 항상 같은 행동을 하는 대신 몇 가지 선택지를 줄 수 있습니다. 예를 들어, 이전 FSM에서 우리의 AI 탱크는 플레이어가 시야에 들어오면 항상 플레이어 탱크를 쫓았습니다(Chase). 대신 '플레이어가 시야에 들어옴' 트랜지션(전환)을 분할하여 도망(Flee)이라는 새로운 추가 상태에 연결할 수 있습니다. AI가 어느 상태로 이동할지 어떻게 결정할 수 있을까요? 물론, 무작위로 합니다:

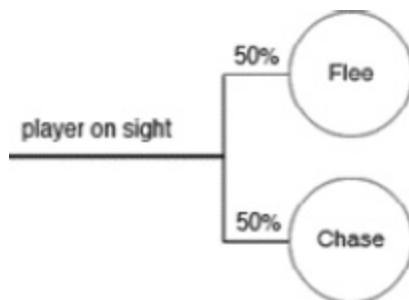


그림 3.9 – 확률을 사용하는 FSM

앞의 다이어그램에서 볼 수 있듯이, 이제 AI 탱크가 플레이어를 발견했을 때 매번 쫓는 대신 50%의 확률로 현장에서 도망칩니다(아마도 본부에 공격을 보고하거나 다른 이유일 것입니다). 이전 주사위 예제에서 했던 것과 같은 방식으로 이 메커니즘을 구현할 수 있습니다. 먼저 1에서 100 사이의 값을 무작위로 구한 다음, 그 값이 1과 50 사이에 있는지, 아니면 51과 100 사이에 있는지 확인합니다. 전자라면 탱크는 도망가고, 그렇지 않으면 플레이어를 쫓을 것입니다.

무작위 선택을 구현하는 또 다른 방법은 룰렛 휠 선택 알고리즘(roulette wheel selection algorithm)을 사용하는 것입니다. 이 알고리즘은 정확한 확률이 없거나 컴파일 시간에 가능한 모든 선택지를 알 수 없는 경우(예를 들어 파일에서 FSM 규칙을 로드하기 때문에) 매우 유리합니다.

이름에서 알 수 있듯이, 각 사건마다 하나의 섹터(영역)가 있는 룰렛 휠을 상상하는 아이디어입니다. 단, 사건이 발생할 확률이 높을수록 섹터가 커집니다. 그런 다음 수학적으로 휠을 돌려 멈춘 섹터에 해당하는 사건을 선택합니다:

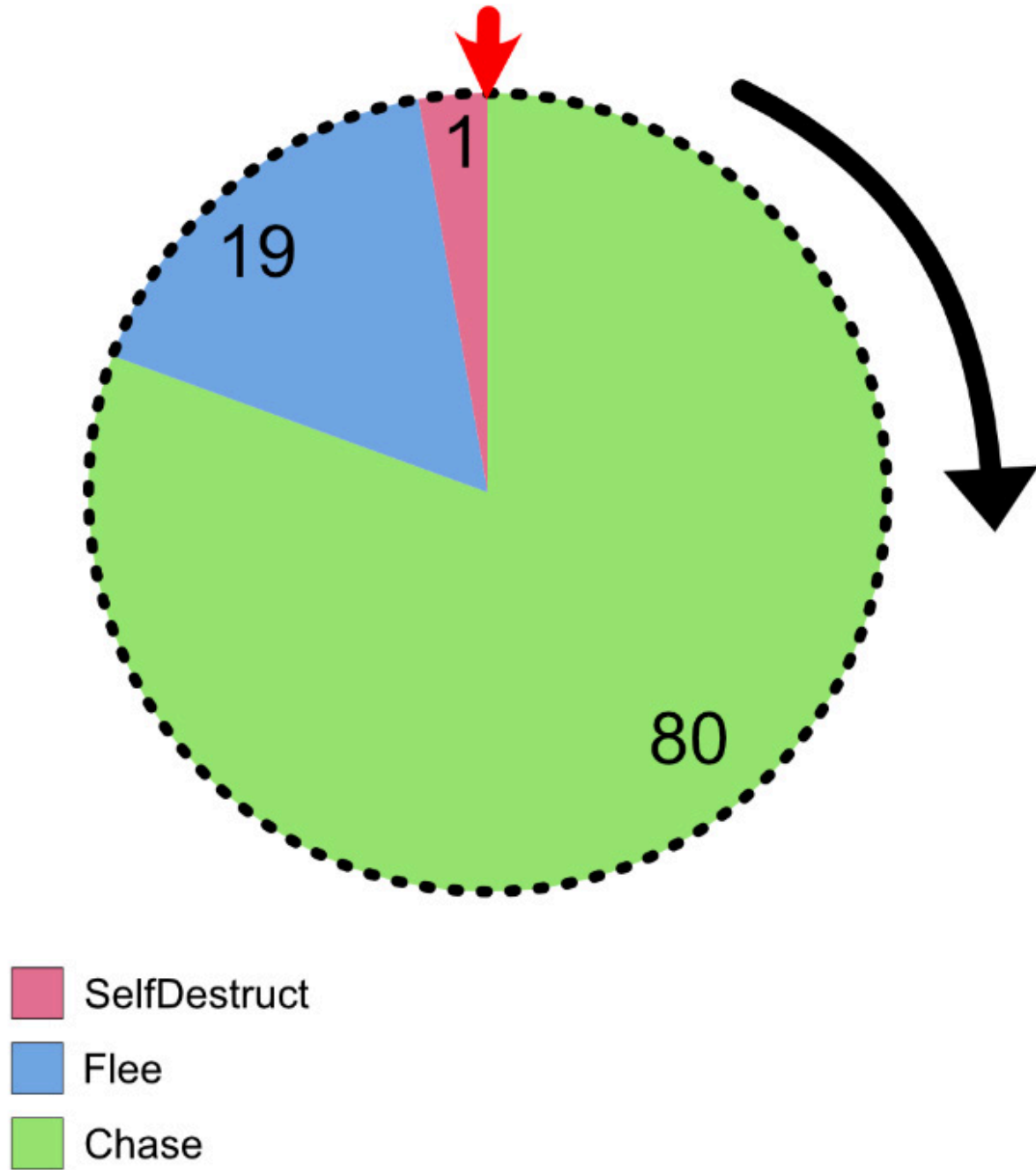


그림 3.10 – 룰렛 휠

우리 예제에는 추적(Chase), 도망(Flee), 자폭(SelfDestruct)이라는 세 가지 상태가 있습니다. 각 상태에 대해 다른 상태들과 비교하여 얼마나 일어날 법한지를 나타내는 가중치(weight)를 할당합니다. 예를 들어 그림에서 볼 수 있듯 Chase에 가중치 80, Flee에 19, SelfDestruct에 1을 설정했습니다. 가중치는 확률처럼 합이 1이 되거나 100이 될 필요는 없으며 특별한 숫자가 될 필요도 없다는 점에 유의하세요.

하지만 이 경우 가중치를 확률로 변환하기 쉽도록 합계가 100이 되게 만들었습니다. Chase가 80%의 확률로, Flee가 19%의 확률로 발생하고, 나머지 1%의 경우 탱크가 자폭한다고 상상할 수 있습니다. 그

러나 일반적으로 X 사건의 가중치를 X라고 적힌 공의 개수로 생각하고 추첨함(lottery box)에 넣는 것으로 상상할 수 있습니다.

FSM.cs 파일에서 결과를 살펴보겠습니다:

```
using UnityEngine;
using System.Collections;
using System;
using System.Linq;

public class FSM : MonoBehaviour {
    [Serializable]
    public enum FSMState {
        Chase,
        Flee,
        SelfDestruct,
    }

    [Serializable]
    public struct FSMProbability {
        public FSMState state;
        public int weight;
    }

    public FSMProbability[] states;

    FSMState selectState() {
        // 모든 상태의 가중치를 합산합니다.
        var weightSum = states.Sum(state => state.weight);
        var randomNumber = UnityEngine.Random.Range(0,
            weightSum);
        var i = 0;
        while (randomNumber >= 0) {
            var state = states[i];
            randomNumber -= state.weight;
            if (randomNumber <= 0) {
                return state.state;
            }
            i++;
        }
        // 이 지점에 도달하는 것은 불가능합니다!
        throw new Exception("selectState 알고리즘에
            원가 문제가 있습니다!");
    }

    // Update는 매 프레임마다 한 번씩 호출됩니다.
```

```

void Update () {
    if (Input.GetKeyDown(KeyCode.Space))
    {
        FSMState randomState = selectState();
        Debug.Log(randomState.ToString());
    }
}
}

```

메커니즘은 매우 간단합니다. 먼저 상상 속 휠의 크기를 알기 위해 모든 가중치를 합산합니다. 그런 다음 0과 이 합계 사이의 숫자를 선택합니다. 마지막으로, 숫자가 음수가 될 때까지 (첫 번째부터 시작하여) 각 상태의 가중치를 이 숫자에서 뺍니다. Update() 메서드에서 볼 수 있듯이, 스페이스바를 누를 때마다 알고리즘이 states 배열에서 임의의 항목을 하나씩 선택합니다.

AI 스킬의 동적 적응(Dynamically adapting AI skills)

확률을 사용하여 AI 캐릭터의 지능 수준이나 전역 게임 설정을 지정할 수도 있습니다. 이는 차례로 게임의 전반적인 난이도에 영향을 미쳐 플레이어에게 게임을 도전적이고 흥미진진하게 유지하게 해줍니다. 제시 셸(Jesse Schell)의 책 *게임 디자인의 예술(The Art of Game Design)*에서 설명된 바와 같이, 플레이어는 게임이 자신을 플로우 채널(flow channel, 미하이 칙센트미하이의 몰입 상태에 대한 심리학적 연구에서 게임으로 각색된 개념)에 유지시켜줄 때만 계속해서 게임을 플레이합니다:

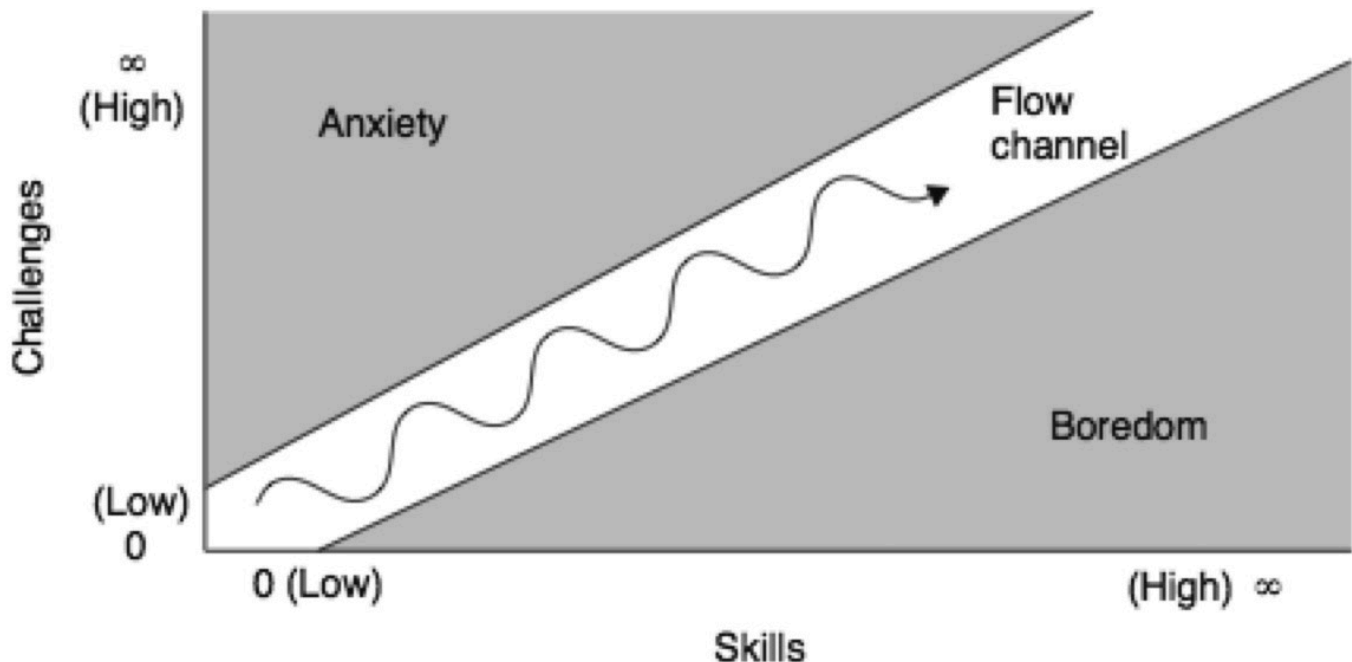


그림 3.11 – 플레이어의 플로우 채널

플레이어가 필요한 기술을 갖추기 전에 너무 까다로운 도전을 제시하면, 그들은 불안감을 느끼고 실망할 것입니다. 반면에 게임을 마스터한 후에도 동일한 페이스를 유지한다면 지루해질 것입니다. 플레이어가 오랫동안 몰입(engaged)을 유지하는 영역은 이 어렵고 쉬운 양극단 사이에 있으며, 원작자는 이를 플로우 채널이라고 불렀습니다. 플레이어를 플로우 채널에 유지하기 위해, 게임 디자이너는 플레이어가 시간

이 지남에 따라 획득하는 증가하는 기술에 맞는 도전과 임무를 제공해야 합니다. 하지만 학습 속도와 기 대치는 개인마다 다를 수 있기 때문에 모든 플레이어에게 효과가 있는 값을 찾는 것은 쉽지 않습니다.

이 문제를 해결하는 한 가지 방법은 게임플레이 세션 중에 플레이어의 시도와 결과를 수집하고, 그에 따 라 상대 AI의 난이도를 조정하는 것입니다. 그렇다면 AI의 난이도를 어떻게 변경할 수 있을까요? 예를 들어 AI를 더 공격적으로 만들거나, 완벽한 공격을 맞출 확률을 높이거나, 비정상적인 행동(erratic behavior) 확률을 낮추는 방식으로 할 수 있습니다.

슬롯머신 만들기

이 데모에서는 10개의 심벌(symbols)과 3개의 릴(reels)이 있는 슬롯머신 게임을 설계하고 구현합니다. 간단하게 만들기 위해 0에서 9까지의 숫자를 심벌로 사용하겠습니다. 많은 슬롯머신이 과일이나 종, 별, 글자와 같은 다른 단순한 모양을 사용합니다. 일부 다른 슬롯머신은 인기 영화나 TV 프랜차이즈를 기반 으로 한 특정 테마를 사용합니다. 10개의 심벌과 3개의 릴이 있으므로 가능한 조합은 총 1,000가지 (10^3)입니다.

랜덤 슬롯머신

이 무작위 슬롯머신 데모는 이전 주사위 예제와 비슷합니다. 이번에는 3개의 릴에 대해 3개의 난수를 생 성할 것입니다. 유일한 지불금(payout)은 페이 라인(pay line, 지불 선)에 동일한 심벌 3개가 나올 때뿐 입니다. 더 간단하게 만들기 위해 이 데모에서는 플레이할 라인을 단 하나만 갖습니다. 플레이어가 이기 면 베팅 금액의 500배를 돌려줍니다.

모든 UI 요소로 씬을 설정하겠습니다. 릴을 위한 3개의 텍스트, YOU WIN 또는 YOU LOSE 텍스트를 위한 또 다른 텍스트 요소(betResult 오브젝트), 플레이어의 크레딧을 위한 하나의 텍스트 요소 (Credits), 베팅을 위한 입력 필드(InputField), 그리고 레버를 당기기 위한 버튼(Button)입니다.

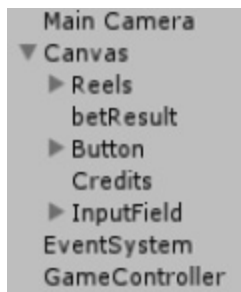


그림 3.12 – 우리의 GUI 텍스트 오브젝트

새로운 스크립트의 모습은 다음 SlotMachine.cs 파일과 같습니다:

```
using UnityEngine;
using UnityEngine.UI;

public class SlotMachine : MonoBehaviour {
    public float spinDuration = 2.0f;
    public int numberOfSym = 10;
```



```

public Text firstReel;
public Text secondReel;
public Text thirdReel;
public Text betResult;
public Text totalCredits;
public InputField inputBet;

private bool startSpin = false;
private bool firstReelSpinned = false;
private bool secondReelSpinned = false;
private bool thirdReelSpinned = false;

private int betAmount;
private int credits = 1000;
private int firstReelResult = 0;
private int secondReelResult = 0;
private int thirdReelResult = 0;
private float elapsedTime = 0.0f;

```

먼저 필요한 모든 클래스 속성을 나열하는 것으로 시작합니다. 다시 말하지만, 엄격하게 필요하지 않은 `public` 필드를 피하는 것이 좋은 프로그래밍 관행이라는 점에 유의하세요. 따라서 대신 `[SerializeField]` 속성을 사용해야 합니다. 하지만 여기서는 코드 길이가 너무 길어지는 것을 피하기 위해 `public` 속성을 사용하겠습니다.

이제 3개의 새로운 함수를 추가하여 계속 진행하겠습니다. 슬롯머신의 회전을 시작하는 `Spin`, 사용자 인터페이스를 업데이트하는 데 사용할 `OnGui`, 그리고 스핀 결과를 확인하고 플레이어에게 승패를 알려주는 `checkBet` 함수입니다:

```

public void Spin() {
    if (betAmount > 0) {
        startSpin = true;
    } else {
        betResult.text = "유효한 베팅 금액을 입력하세요!";
    }
}

private void OnGUI() {
    try {
        betAmount = int.Parse(inputBet.text);
    } catch {
        betAmount = 0;
    }
    totalCredits.text = credits.ToString();
}

```

```

void checkBet() {
    if (firstReelResult == secondReelResult &&
        secondReelResult == thirdReelResult) {
        betResult.text =
            "YOU WIN!"; credits += 500*betAmount;
    } else {
        betResult.text = "YOU LOSE!"; credits -= betAmount;
    }
}

```

다음으로 스크립트의 메인 루프를 구현합니다. FixedUpdate 함수에서 각 릴을 차례대로 회전시켜 슬롯 머신을 실행합니다. 처음에는 firstReelSpinned, secondReelSpinned, thirdReelSpinned가 모두 false입니다. 따라서 첫 번째 if 블록으로 들어갑니다. 여기서 릴을 무작위 값으로 설정하고 함수를 종료합니다. 일정 시간이 지날 때까지 이 과정을 반복합니다.

그 후 릴을 최종 값으로 설정하고 firstReelSpinned를 true로 설정합니다. 그러면 함수가 두 번째 릴로 이동하여 이 단계를 반복합니다. 마지막으로 세 번째 릴이 최종적으로 최종 값으로 설정된 후 checkBet으로 결과를 확인합니다:

```

void FixedUpdate () {
    if (startSpin) {
        elapsedTime += Time.deltaTime;
        int randomSpinResult =
            Random.Range(0, numberOfSym);

        if (!firstReelSpinned) {
            firstReel.text = randomSpinResult.ToString();
            if (elapsedTime >= spinDuration) {
                firstReelResult = randomSpinResult;
                firstReelSpinned = true;
                elapsedTime = 0;
            }
        } else if (!secondReelSpinned) {
            secondReel.text = randomSpinResult.ToString();
            if (elapsedTime >= spinDuration) {
                secondReelResult = randomSpinResult;
                secondReelSpinned = true;
                elapsedTime = 0;
            }
        } else if (!thirdReelSpinned) {
            thirdReel.text = randomSpinResult.ToString();
            if (elapsedTime >= spinDuration) {
                thirdReelResult = randomSpinResult;
                startSpin = false;
                elapsedTime = 0;
            }
        }
    }
}

```

```

        firstReelSpinned = false;
        secondReelSpinned = false;
        checkBet();
    }
}
}

```

스크립트를 빈 GameController 오브젝트에 연결한 다음 인스펙터에서 참조된 오브젝트를 채웁니다. 그런 다음 Button을 Spin() 메서드에 연결해야 합니다. 그러려면 Button을 선택하고 다음 스크린샷과 같이 인스펙터의 On Click () 이벤트를 핸들러를 채웁니다:

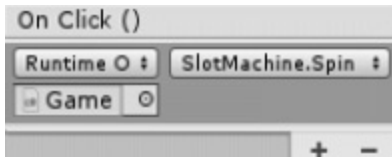


그림 3.13 – OnClick() 이벤트 핸들러

버튼을 클릭하면 startSpin 플래그를 true로 설정합니다. 한 번 회전이 시작되면 FixedUpdate() 메서드에서 각 릴에 대한 무작위 값을 생성합니다. 마지막으로 세 번째 릴의 값을 얻으면 startSpin 플래그를 다시 false로 재설정합니다. 각 릴에 대해 무작위 값을 얻는 동안, 플레이어가 레버를 당긴 후 경과된 시간도 추적합니다.

보통 실제 슬롯머신에서는 결과를 내기 전에 각 릴이 3~5초 정도 소요됩니다. 따라서 여기에서도 최종 무작위 값을 표시하기 전에 spinDuration에 지정된 대로 약간의 시간을 둡니다. 싹을 플레이하고 'Pull Lever' 버튼을 클릭하면 다음 스크린샷과 같은 최종 결과를 볼 수 있습니다:

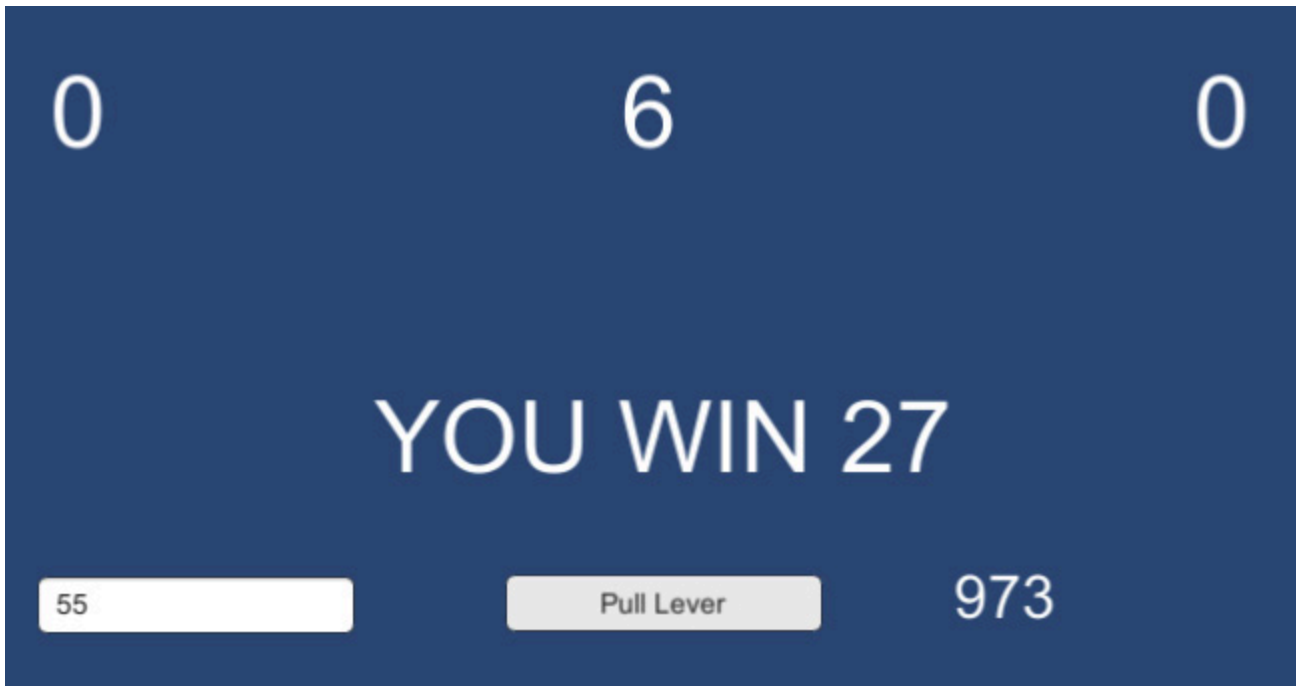


그림 3.14 – 작동 중인 우리의 랜덤 슬롯 게임

당첨 확률이 1/100이기 때문에 연속해서 여러 번 지다 보면 금방 지루해집니다. 하지만 슬롯머신을 한 번이라도 플레이해 본 적이 있다면, 그것이 실제 작동 방식이 아니라는 것을, 혹은 적어도 더 이상은 그렇지 않다는 것을 알 것입니다. 보통 플레이하는 동안 여러 번의 승리를 거둘 수 있습니다. 이러한 작은 승리가 원금 베팅을 모두 회수해 주지는 못하지만(그리고 장기적으로 대부분의 플레이어는 파산하지만), 슬롯머신은 여전히 때때로 승리 그래픽과 흥미진진한 사운드를 렌더링하며, 연구자들은 이를 승리로 위장된 패배(losses disguised as wins)라고 부릅니다.

따라서 잭팟을 터뜨리는 단 한 가지 방법만 두는 대신, 플레이 세션 동안 슬롯머신이 더 작은 수익을 지급하도록 규칙을 약간 수정해 보겠습니다.

가중 확률 (Weighted probability)

실제 슬롯머신에는 기계의 전체 설계 문서인 페이테이블 및 릴 스트립(PARS, Paytable and Reel Strips) 시트라는 것이 있습니다. PARS 시트는 지불금 비율, 승리 패턴, 그에 따른 지불금 등을 지정하는 데 사용됩니다.

지금 상금의 수와 그 빈도는 하우스(슬롯머신)가 장기적으로 항상 이기도록 세심하게 선택되어야 하며, 동시에 기계를 매력적으로 느끼게 만들기 위해 때때로 플레이어에게 무언가를 돌려주도록 해야 합니다. 이를 환수율 또는 플레이어 환수율(RTP, Return to Player)이라고 합니다. 예를 들어 RTP가 90%인 슬롯머신은 시간이 지남에 따라 기계가 전체 베팅 금액의 평균 90%를 플레이어에게 돌려준다는 의미입니다.

이 데모에서는 시간이 지남에 따라 특정 승리를 산출하기 위해 하우스의 최적 값을 선택하거나 특정 환수율을 유지하는 데 초점을 맞추지 않을 것입니다. 대신 특정 심벌이 평소보다 더 많이 나타나도록 확률에 가중치를 부여하는 방법을 시연하겠습니다. 0 기호가 1번째와 3번째 릴에서 평소보다 20% 더 많이 나타나게 하고 베팅 금액의 절반을 지불금으로 돌려주도록 해 보겠습니다.

다시 말해, 플레이어는 1번째와 3번째 릴에서 숫자 0 기호를 얻으면 베팅 금액의 절반만 잃게 되며, 본질적으로 패배를 작은 승리로 위장합니다. 현재 숫자 0 기호가 나올 확률은 1/10(0.1) 또는 10%입니다. 다음 SlotMachineWeighted.cs 파일에 표시된 것처럼, 이제 첫 번째와 세 번째 릴에 0이 떨어질 확률을 30%로 변경하겠습니다(예제 코드에서 SlotMachineWeighted 컴포넌트로 전환하는 것을 잊지 마세요!).

```
using UnityEngine;
using System.Collections;
using UnityEngine.UI;

public class SlotMachineWeighted : MonoBehaviour {
    public float spinDuration = 2.0f;
    // 슬롯머신 릴의 심벌 수
    public int numberOfSym = 10;
    public Text firstReel;
    public Text secondReel;
    public Text thirdReel;
```

```

public Text betResult;
public Text totalCredits;
public InputField inputBet;

private bool startSpin = false;
private bool firstReelSpinned = false;
private bool secondReelSpinned = false;
private bool thirdReelSpinned = false;

private int betAmount = 100;
private int credits = 1000;

[Serializable]
public struct WeightedProbability {
    public int number;
    public int weight;
}

private List<WeightedProbability> weightedReelPoll =
    new List<WeightedProbability>();
private int zeroProbability = 30;
private int firstReelResult = 0;
private int secondReelResult = 0;
private int thirdReelResult = 0;
private float elapsedTime = 0.0f;

```

1번째와 3번째 릴에 0 기호가 나올 확률 퍼센티지를 지정하기 위해 zeroProbability와 같은 새로운 변수 선언이 추가되었습니다. 예를 들어 zeroProbability가 30이면 3번째 릴은 30%의 확률로 0을 표시합니다. 앞선 FSM 예제에서 했던 것처럼 가중치가 적용된 심벌을 채우기 위해 weightedReelPoll 배열 리스트가 사용됩니다.

그런 다음, 다음 코드에서 보듯이 Start() 메서드에서 이 리스트를 초기화합니다:

```

void Start () {
    weightedReelPoll.Add(new WeightedProbability {
        number = 0,
        weight = zeroProbability
    });

    int remainingValuesProb = (100 - zeroProbability)/9;

    for (int i = 1; i < 10; i++) {
        weightedReelPoll.Add(new WeightedProbability {
            number = i,
            weight = remainingValuesProb
        });
    }
}

```

```
});
}}
```

실제로 0의 값을 30으로 설정하고 남은 70퍼센트 포인트를 나머지 9개 숫자에 나눕니다.

또한 수정되고 개선된 checkBet() 메서드를 작성하고 있습니다. 단 하나의 잭팟 승리 옵션 대신, 이제 잭팟, 승리로 위장된 패배, 아차상(near miss), 첫 번째와 세 번째 행에서 일치하는 임의의 심벌 두 개, 그리고 당연히 패배 조건까지 5가지 조건을 고려하고 있습니다:

```
void checkBet() {
    if (firstReelResult == secondReelResult &&
        secondReelResult == thirdReelResult) {
        betResult.text = "JACKPOT!";
        credits += betAmount * 50;
    } else if (firstReelResult == 0 &&
        thirdReelResult == 0) {
        betResult.text =
            "YOU WIN " + (betAmount/2).ToString();
        credits -= (betAmount/2);
    } else if (firstReelResult == secondReelResult) {
        betResult.text = "Aww... ALMOST JACKPOT!";
    } else if (firstReelResult == thirdReelResult) {
        betResult.text =
            "YOU WIN " + (betAmount*2).ToString();
        credits -= (betAmount*2);
    } else {
        betResult.text = "YOU LOSE!";
        credits -= betAmount;
    }
}
```

**checkBet() 메서드에서 잭팟을 터뜨리면 베팅 금액의 50배를 돌려받고, 1번째와 3번째 릴이 0이면 베팅 금액의 50%를 잃으며, 1번째와 3번째 릴이 다른 어떤 심벌과 일치하더라도 두 배를 따도록 슬롯머신을 설계했습니다.

그런 다음 이전 예제와 마찬가지로 다음 코드와 같이 FixedUpdate() 메서드에서 3개의 릴에 대한 값을 생성합니다:

```
private int PickNumber() {
    // 모든 상태의 가중치를 합산합니다.
    var weightSum =
        weightedReelPoll.Sum(state => state.weight);
    var randomNumber =
        UnityEngine.Random.Range(0, weightSum);
```

```

var i = 0;
while (randomNumber >= 0) {
    var candidate = weightedReelPoll[i];
    randomNumber -= candidate.weight;
    if (randomNumber <= 0) {
        return candidate.number;
    }
    i++;
}
// 이 지점에 도달해서는 안 됩니다!
throw new Exception("selectState 알고리즘에
                      뭔가 문제가 있습니다!");
}

void FixedUpdate () {
    if (startSpin) {
        elapsedTime += Time.deltaTime;
        int randomSpinResult =
            Random.Range(0, numberOfSym);

        if (!firstReelSpinned) {
            firstReel.text = randomSpinResult.ToString();
            if (elapsedTime >= spinDuration) {
                int weightedRandom = PickNumber();
                firstReel.text = weightedRandom.ToString();
                firstReelResult = weightedRandom;
                firstReelSpinned = true;
                elapsedTime = 0;
            }
        } else if (!secondReelSpinned) {
            secondReel.text = randomSpinResult.ToString();
            if (elapsedTime >= spinDuration) {
                secondReelResult = randomSpinResult;
                secondReelSpinned = true;
                elapsedTime = 0;
            }
        }
    }
}
...

```

첫 번째 릴의 경우 회전 기간 동안 발생하는 실제 무작위 값을 보여줍니다. 시간이 다 되면 확률 분포에 따라 심벌로 이미 채워진 폴(poll)에서 값을 선택합니다. 따라서 0 기호는 나머지 기호들보다 30% 더 발생할 가능성이 높습니다.

실제로 플레이어가 1번째와 3번째 릴에서 두 개의 0 기호를 얻으면 베팅 금액을 잃게 되지만, 우리는 이를 승리처럼 보이게 만듭니다. 여기서는 그저 허접한 메시지일 뿐이지만, 멋진 그래픽, 불꽃놀이, 멋진

승리 효과음과 결합한다면 효과가 있을 수 있습니다.

아차상 / 니어 미스 (A near miss)

첫 번째와 두 번째 릴에서 동일한 심벌이 나오면, 두 번째 릴과 가까운 숫자를 세 번째 릴에 무작위 값으로 반환하여 플레이어에게 아차상 효과(near-miss effect)를 제공해야 합니다. 세 번째 무작위 스핀 결과를 먼저 확인하여 이를 수행할 수 있습니다. 무작위 값이 첫 번째 및 두 번째 결과와 동일하다면 잭팟이므로 결과를 변경해서는 안 됩니다.

하지만 그렇지 않다면, 결과를 다른 두 결과와 충분히 가깝게 수정해야 합니다. 다음 코드의 주석을 확인하세요:

```
else if (!thirdReelSpinned) {
    thirdReel.text = randomSpinResult.ToString();
    if (elapsedTime >= spinDuration) {
        if ((firstReelResult == secondReelResult)
            && randomSpinResult != firstReelResult) {
            // 처음 두 릴이 같은 심벌을 냈지만
            // 안타깝게도 세 번째 릴이 빗나갔습니다.
            // 따라서 무작위 숫자를 주는 대신
            // 다른 두 숫자보다 1 적거나 많은
            // 심벌을 반환할 것입니다.
            randomSpinResult = firstReelResult - 1;
            if (randomSpinResult < firstReelResult)
                randomSpinResult =
                    firstReelResult - 1;
            if (randomSpinResult > firstReelResult)
                randomSpinResult =
                    firstReelResult + 1;
            if (randomSpinResult < 0)
                randomSpinResult = 0;
            if (randomSpinResult > 9)
                randomSpinResult = 9;

            thirdReel.text =
                randomSpinResult.ToString();
            thirdReelResult = randomSpinResult;

        } else {
            int weightedRandom = PickNumber();
            thirdReel.text =
                weightedRandom.ToString();
            thirdReelResult = weightedRandom;
        }
    }
    startSpin = false;
```



```

        elapsedTime = 0;
        firstReelSpinned = false;
        secondReelSpinned = false;
        checkBet();
    }
}
}

```

그리고 그 아차상(near miss)이 발생하면 다음 스크린샷과 같은 결과를 보게 될 것입니다:

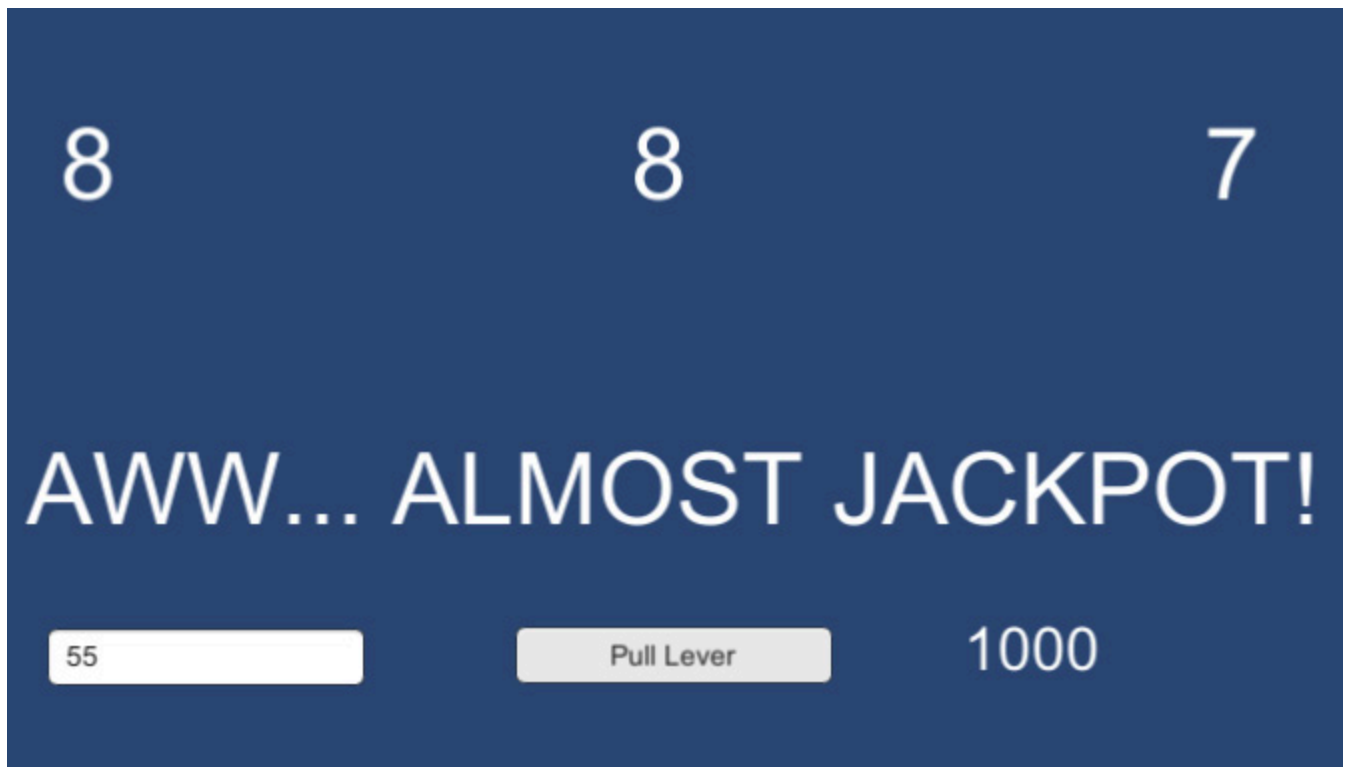


그림 3.15 – 아차상 (니어 미스)

베팅 금액에 따라 실시간으로 확률을 조정함으로써 한 걸음 더 나아갈 수도 있습니다(하지만 그것은 너무 비양심적이겠죠). 마지막으로 플레이어가 돈을 모두 잃었을 때 나타나는 게임 오버(Game Over) 메시지를 추가할 수 있습니다.

이 데모는 슬롯머신 게임의 기본 구현을 보여줍니다. 이 뼈대에서 시작하여 더 멋진 그래픽, 애니메이션, 사운드 효과로 개선할 수 있습니다. 하지만 여기서 얻어야 할 중요한 교훈은, 무작위성과 확률만으로도 이미 게임을 만들 수 있다는 것을 이해하는 것입니다.

요약

이 장에서는 AI 게임 디자인에서의 확률 응용에 대해 배웠습니다. 유니티 3D에서 이를 구현하여 몇 가지 기법을 실험해 보았습니다. 보너스로 슬롯머신이 어떻게 작동하는지도 배웠고 유니티 3D를 사용하여 간

단한 슬롯머신 게임을 구현했습니다. 게임에서의 확률은 플레이어가 결과를 예측할 수 없도록 행동에 불확실성을 추가하여 게임과 캐릭터를 더 사실적으로 보이게 만드는 것입니다.

다음 장에서는 센서를 구현하는 방법과 센서를 통해 AI가 주변 환경을 인식하게 만드는 방법을 살펴보겠습니다.

추가 자료

베이즈 기법(Bayesian techniques)을 사용한 불확실성 하에서의 의사 결정과 같이 게임 확률에 관한 고급 기법을 더 공부하고 싶다면, 데이비드 M. 버그(David M. Bourg)와 글렌 씨먼(Glenn Seeman)이 쓴 *AI for Game Developers*를 읽어볼 것을 권장합니다. 케이티 살렌(Katie Salen)의 *Rules of Play* 역시 게임 디자인에 관해 추천하는 책입니다.